



l'école d'ingénierie
informatique

Rapport d'activité de stage

BTS - SIO Option Solutions Logicielles et Applications Métier
(SLAM)

Entreprise d'accueil : FORMATINFO (Castelnau-le-Lez)

Tuteur de stage : M. Lilian ANNET (Directeur)

Période du stage : Du 05 janvier au 27 février 2026

Présenté par : ROUSTAN-LABOURET Albin

Campus : EPSI Montpellier

Année scolaire : 2025 - 2026

I. Remerciements.....	3
II. Introduction.....	3
III. Présentation de l'Entreprise et Contexte du Projet (Corrigé).....	4
1. Présentation de FORMATINFO.....	4
2. Contexte et définition de la mission.....	4
IV. Méthodologie, Outils et Sécurité.....	5
1. L'environnement de développement et les outils.....	5
2. Gestion de projet et méthodologie de travail.....	5
3. Expression des besoins et Cas d'Utilisation (UML).....	5
4. Sécurisation de l'application (Cybersécurité).....	7
V. Analyse et Conception.....	8
1. De l'expression du besoin à la modélisation.....	8
2. Modélisation des interactions utilisateurs et nomenclature.....	10
VI. Développement Backend et Base de Données.....	11
1. L'implémentation technique et le SQL.....	11
2. Gestion des enfants et des contraintes.....	12
3. Débogage et résolution d'incidents.....	12
4. Insertion des jeux d'essais et requêtes complexes.....	13
VII. Développement Frontend et Optimisation.....	14
1. L'enjeu de l'interface et du référencement naturel (SEO).....	14
2. L'intégration technique du SEO via PHP.....	15
3. Conception des pages de façade et hiérarchie sémantique.....	16
4. L'affichage dynamique des médias.....	17
VIII. Bilan, Compétences et Conclusion.....	18
1. Bilan technique et compétences mobilisées.....	18
2. Bilan professionnel et humain.....	18
3. Conclusion et perspectives.....	19
Annexes.....	20

I. Remerciements

Je tiens tout d'abord à remercier Monsieur Lilian ANNET, fondateur et dirigeant de FORMATINFO, pour m'avoir accueilli au sein de son entreprise durant ces huit semaines.

Travailler en lien direct avec un professionnel indépendant a été une véritable chance. Sa confiance, sa disponibilité et la pertinence du projet technique qu'il m'a confié m'ont permis d'évoluer dans un cadre à la fois autonome et extrêmement formateur, avec des échanges techniques rapides et efficaces.

Enfin, je remercie l'équipe pédagogique de l'EPSI Montpellier pour la qualité des enseignements dispensés tout au long de l'année, qui m'ont fourni les bases théoriques (notamment en SQL et PHP) nécessaires à la réussite de ces missions.

II. Introduction

Dans le cadre de ma formation en BTS SIO (Services Informatiques aux Organisations) à l'EPSI Montpellier, j'ai eu l'opportunité de réaliser un stage de développement au sein de l'entreprise FORMATINFO, située à Castelnau-le-Lez. Ce stage, d'une durée de huit semaines, s'inscrit dans une volonté de mettre en pratique les compétences acquises durant mon cursus, particulièrement dans la conception d'architectures logicielles et le développement web backend.

Le développement d'applications web modernes nécessite aujourd'hui une gestion de données de plus en plus complexe. C'est dans ce contexte que m'a été confiée la mission principale de ce stage : concevoir et développer le socle technique d'une plateforme de Vidéo à la Demande (VOD), fortement inspirée des standards actuels du marché du streaming.

Ce projet ne se limitait pas à la simple création d'une interface graphique de présentation. Le véritable enjeu, sur lequel j'ai concentré la majorité de mes efforts, résidait dans la conception d'un moteur de base de données robuste et évolutif. Il fallait créer un système capable de faire la distinction hiérarchique entre différents types de médias (films uniques et séries à épisodes), tout en gérant de manière fluide les interactions des utilisateurs (reprise de lecture précise, favoris, notations).

Ce rapport détaille les choix architecturaux retenus, notamment la mise en place d'un modèle de base de données par héritage, les obstacles techniques surmontés lors du développement, et les solutions algorithmiques mises en œuvre pour lier le backend à l'interface utilisateur.

III. Présentation de l'Entreprise et Contexte du Projet (Corrigé)

1. Présentation de FORMATINFO

FORMATINFO est une entreprise individuelle spécialisée dans les services informatiques, implantée à Castelnau-le-Lez. Gérée par Monsieur Lilian ANNET, auto-entrepreneur, elle accompagne ses clients dans la numérisation de leurs outils et le développement de solutions logicielles sur-mesure.

La structure en auto-entreprise offre une flexibilité et une réactivité maximales. Outre ses prestations clients, FORMATINFO maintient une veille technologique constante et favorise le développement de projets internes (R&D) pour tester de nouvelles architectures logicielles et renforcer son expertise technique. C'est dans ce cadre d'innovation interne que mon stage s'est déroulé, m'offrant l'opportunité de collaborer en direct avec le dirigeant, sans intermédiaire.

2. Contexte et définition de la mission

Le projet de développement d'une plateforme de streaming a été initié pour éprouver des concepts avancés de modélisation de données et de développement full-stack. L'objectif technique était de concevoir un produit fini ou, à minima, un "Proof of Concept" (Preuve de concept) très avancé.

Le cahier des charges exigeait la réalisation des éléments suivants :

- **Backend et Base de données (Priorité absolue)** : Construire un système relationnel sans faille, permettant des requêtes rapides sur un large volume de données hétérogènes (films, séries, épisodes, acteurs, catégories).
- **Logique Métier** : Développer une logique backend capable de lier intelligemment la base de données à l'affichage, incluant un système de routage automatisé pour les images (système de slug).
- **Frontend** : Développer des pages de façade dynamiques permettant la navigation dans le catalogue et la mise en avant des œuvres.

IV. Méthodologie, Outils et Sécurité

1. L'environnement de développement et les outils

Pour mener à bien la création de cette plateforme de A à Z, j'ai dû mettre en place un environnement de développement professionnel complet. Travaillant en autonomie, il était crucial d'avoir des outils fiables pour coder, tester et versionner mon travail.

- **Serveur Local (XAMPP / WAMP)** : Afin de simuler l'environnement de production, j'ai utilisé une pile logicielle locale (Apache, MySQL, PHP). Cela m'a permis de tester mes requêtes en temps réel et de visualiser l'interface sans nécessiter d'hébergement web externe lors de la phase de développement.
- **Éditeur de code (IDE)** : J'ai utilisé Visual Studio Code. Ses extensions dédiées à PHP (IntelliSense) et au formatage HTML/CSS ont considérablement accéléré la saisie du code et réduit les erreurs de syntaxe.
- **Versionning (Git & GitHub)** : Bien que travaillant seul sur le code, j'ai mis un point d'honneur à utiliser Git. Cela m'a permis de sauvegarder l'historique de mes modifications, de créer des "branches" pour tester de nouvelles fonctionnalités (comme l'ajout du système de favoris) sans casser la branche principale (`main`), et d'assurer une sauvegarde externalisée du projet.

2. Gestion de projet et méthodologie de travail

Le cadre de la mission, en lien direct avec l'auto-entrepreneur de FORMATINFO, a favorisé l'adoption d'une méthodologie agile, bien qu'adaptée à un développeur unique.

Au lieu de rédiger un cahier des charges figé de dizaines de pages, nous avons opté pour des cycles de développement courts (sprints) et des validations régulières.

J'ai utilisé un Trello pour organiser mes tâches en trois colonnes : "À faire", "En cours", "Terminé".

Cette organisation m'a permis d'être transparent sur mon avancement auprès de la direction de FORMATINFO et d'ajuster les priorités en cours de route, notamment lorsque la restructuration de la base de données s'est avérée plus complexe que prévu.

3. Expression des besoins et Cas d'Utilisation (UML)

Avant de concevoir la base de données, il a fallu définir précisément ce que les utilisateurs pourraient faire sur l'application. J'ai modélisé ces besoins à l'aide d'un diagramme des cas d'utilisation (UML).

L'application distingue deux types d'acteurs principaux :

Le Visiteur (Non authentifié) : Il peut naviguer sur la page d'accueil, consulter le catalogue (films et séries) et lire les fiches descriptives, mais il n'a pas accès au lecteur vidéo ni aux fonctionnalités interactives.

L'Utilisateur (Membre authentifié) : En plus des droits du visiteur, il peut lancer une vidéo, reprendre sa lecture en cours (*Progress*), ajouter une œuvre à sa liste de favoris et laisser une note.

L'analyse fonctionnelle de la plateforme s'est articulée autour des deux acteurs majeurs définis dans l'expression du besoin : le Visiteur et l'Utilisateur authentifié. Le Diagramme de Cas d'Utilisation modélise l'ensemble des interactions possibles entre ces acteurs et le système, en distinguant clairement les fonctionnalités de consultation de celles qui nécessitent une persistance de données.

Description des Cas d'Utilisation

Cas d'utilisation de consultation (Acteur : Visiteur et Utilisateur)

Ces cas d'utilisation sont ouverts à tous et couvrent les besoins de navigation et de découverte du catalogue :

- **Naviguer sur la plateforme :** Permet la consultation de la page d'accueil et des différentes sections (par exemple, les catégories).
- **Consulter le catalogue :** Permet la recherche et le filtrage des médias (films et séries) disponibles.
- **Afficher la fiche média :** Permet la lecture des métadonnées (description, acteurs, date de sortie) pour un média spécifique.

Cas d'utilisation interactifs (Acteur : Utilisateur Authentifié)

Ces fonctionnalités constituent la logique métier complexe du backend et sont soumises à une vérification de session :

- **Authentification :** Permet à l'utilisateur d'accéder à son compte personnel, ce qui inclut les processus d'inscription et de connexion. Ce cas d'utilisation est le prérequis de toutes les actions suivantes.
- **Lire un média :** Permet le lancement du lecteur vidéo. Cette fonctionnalité est liée de manière conditionnelle à la table *progress* pour permettre la reprise de lecture.
- **Gérer les Favoris :** Permet d'ajouter ou de retirer un média de sa liste personnelle. Cette action crée ou supprime un enregistrement dans la table *favorites*.
- **Noter un média :** Permet l'enregistrement d'une évaluation par l'utilisateur, impactant la note globale affichée sur la fiche média. L'enregistrement est sécurisé pour garantir une seule note par utilisateur et par média.
- **Reprendre la lecture (*Progress*) :** Un cas d'utilisation étendu à *Lire un média* qui enregistre le *timestamp* de l'arrêt de lecture. Pour les séries, il inclut la gestion de l'ID d'épisode précis.

La modélisation UML a permis de valider que la conception de la base de données (notamment les tables *users*, *favorites*, *votes*, et *progress*) répondait de manière exhaustive aux besoins fonctionnels définis, tout en assurant que les requêtes et les jointures seraient ciblées et performantes.

4. Sécurisation de l'application (Cybersécurité)

La création d'une plateforme gérant des utilisateurs impose une rigueur absolue en matière de cybersécurité. N'utilisant pas de framework gérant la sécurité nativement, j'ai dû implémenter moi-même les protections contre les failles web les plus courantes de l'OWASP.

Protection contre les Injections SQL

Pour interagir avec la base de données, j'ai exclusivement utilisé l'extension PHP Data Objects (PDO) avec des requêtes préparées.

Extrait de code : Vérification des identifiants à la connexion

PHP

```
// Connexion sécurisée avec requête préparée
$stmt = $pdo->prepare('SELECT * FROM users WHERE email = :email');
$stmt->execute(['email' => $post_email]);
$user = $stmt->fetch();
```

Cette méthode empêche un attaquant de manipuler la requête SQL via les champs de formulaire, car les données envoyées sont traitées comme du texte brut et non comme des commandes exécutables par le serveur MySQL.

Hachage des mots de passe

Conformément aux directives de la CNIL et du RGPD, aucun mot de passe n'est stocké en clair dans la base de données. Lors de l'inscription, le mot de passe est "haché" à l'aide de l'algorithme natif et sécurisé de PHP.

PHP

```
// Hachage du mot de passe avant insertion en BDD
$hashed_password = password_hash($user_password,
PASSWORD_BCRYPT);
```

Lors de la connexion, la fonction `password_verify()` est utilisée pour comparer le mot de passe saisi avec l'empreinte stockée, garantissant que même en cas de fuite de la base de données, les mots de passe des utilisateurs restent illisibles.

Protection contre les failles XSS (Cross-Site Scripting)

Afin d'éviter qu'un utilisateur malveillant n'injecte du code JavaScript via d'éventuels champs de saisie (comme un espace commentaire ou un champ de recherche), toutes les données provenant de la base de données ou des formulaires sont systématiquement "échappées" lors de l'affichage frontend grâce à la fonction `htmlspecialchars()`.

V. Analyse et Conception

1. De l'expression du besoin à la modélisation

Une démarche de création "De A à Z"

L'une des spécificités de ce stage a été la prise en charge intégrale du projet, de sa phase d'idéation jusqu'à son déploiement local. Contrairement à l'utilisation d'un CMS (Content Management System) préexistant, le choix s'est porté sur un développement natif (from scratch). Cette démarche "de A à Z" était essentielle pour répondre précisément aux besoins d'une plateforme de streaming, qui nécessite un contrôle total sur les flux de données et la structure de la base.

Avant la moindre ligne de code, la première étape a consisté à analyser la structure des données d'un catalogue multimédia. L'objectif était d'éviter les redondances et de garantir des performances optimales lors des requêtes SQL.

La problématique de la modélisation des médias

Le défi technique majeur résidait dans la nature hétérogène du catalogue. Initialement, la réflexion s'orientait vers une table unique `videos` pour stocker l'ensemble des contenus. Cependant, cette approche montrait rapidement ses limites structurelles :

- Un **film** est une entité unique, directement associée à un fichier vidéo (.mp4) et à une durée.
- Une **série** est un conteneur complexe. Elle ne possède pas de fichier vidéo propre, mais regroupe de multiples épisodes, eux-mêmes répartis en saisons.

Il fallait donc trouver un modèle relationnel capable de stocker ces deux entités sans créer de cases vides (NULL) inutiles dans la base de données.

L'architecture retenue : Le Double Héritage

Pour résoudre ce problème de modélisation, j'ai fait le choix d'implémenter une architecture avancée de **"Class Table Inheritance" (Héritage par tables de classes)**. Cette solution, couramment utilisée dans la conception de bases de données professionnelles, permet de séparer les données communes des données spécifiques.

Le modèle s'articule autour d'une entité parente et de deux entités enfants :

- **La table parente *media*** : Elle constitue le tronc commun. Elle centralise toutes les métadonnées affichées sur la page d'accueil (titre, description, date de sortie, type). C'est également cette table qui gère les relations avec les acteurs, les réalisateurs et les catégories.
- **La table enfant *films*** : Elle hérite de *media* via la clé primaire *media_id* (qui agit également comme clé étrangère) et stocke les spécificités d'un film (le nom du fichier vidéo exact).
- **La table enfant *series*** : Elle hérite également de *media* et permet de faire la liaison avec la table *episodes*, qui contient la nomenclature stricte des saisons et des numéros.

Grâce à cette architecture, le système est évolutif : si l'entreprise souhaite ajouter des documentaires ou des podcasts à l'avenir, il suffira de créer une nouvelle table enfant liée à *media* sans casser l'existant.

Schéma Relationnel de la Base de Données (MLD)

Afin de visualiser clairement cette architecture d'héritage et les différentes relations (One-to-Many, Many-to-Many), j'ai conçu le Modèle Logique de Données (MLD) suivant.

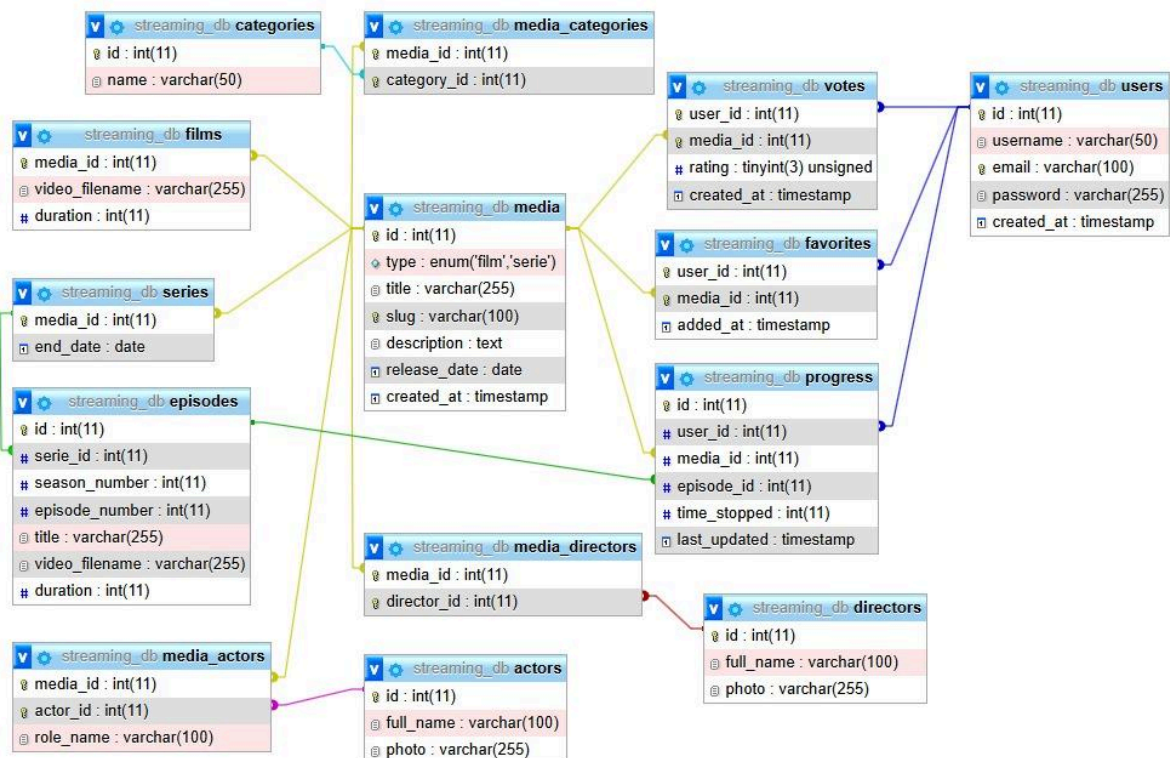


Figure 1 : Schéma relationnel de la base de données illustrant l'héritage de la table *media* vers *films* et *series*, ainsi que les tables d'interaction utilisateur.

Ce schéma met en évidence plusieurs points cruciaux du développement :

- **L'optimisation des liaisons** : Lier un acteur à une série se fait une seule fois sur la table `media`, évitant ainsi de le lier individuellement à des dizaines d'épisodes.
- **La sécurité des utilisateurs** : La table `users` intègre un champ `password` dimensionné en `VARCHAR(255)` pour anticiper le hachage cryptographique des mots de passe en PHP (`password_hash`).

2. Modélisation des interactions utilisateurs et nomenclature

Favoris, Votes et Reprise de lecture

Créer la plateforme de A à Z impliquait également de coder la logique comportementale des utilisateurs. Le modèle d'héritage a permis de simplifier la structure de ces interactions (visibles en bleu et vert sur le schéma) :

- Les tables `favorites` et `votes` ciblent l'ID global du média. L'utilisateur interagit avec l'œuvre dans son ensemble.
- La table `progress` (reprise de lecture) intègre une logique conditionnelle : elle pointe vers le `media_id`, mais possède une colonne `episode_id` nullable. Pour un film, cette valeur reste nulle ; pour une série, elle enregistre l'ID précis de l'épisode, permettant au backend PHP de relancer la vidéo au bon endroit.

Le système de routage dynamique (Le "Slug")

Plutôt que d'alourdir la base de données en y stockant les chemins d'accès complets de chaque image (affiches verticales et fonds horizontaux), j'ai conçu un système de génération dynamique.

La table `media` possède une colonne technique nommée `slug` (ex: `star_wars_3`). Une nomenclature stricte des fichiers a été définie lors de l'analyse : le backend PHP utilise ce slug pour déduire automatiquement le chemin des ressources dans les dossiers du serveur (ex: `/assets/img/vertical/star_wars_3.jpg`). Ce choix technique réduit la taille de la base et facilite la maintenance du catalogue.

VI. Développement Backend et Base de Données

1. L'implémentation technique et le SQL

Choix technologiques et d'environnement

Le développement du backend a été réalisé en PHP, avec une base de données MySQL gérée via l'interface phpMyAdmin. L'environnement de développement local a été configuré pour simuler un serveur web complet. Le choix de développer en PHP natif (sans framework dans un premier temps) a été fait pour garantir une compréhension totale des mécanismes de requêtes et de jointures nécessaires à l'architecture d'héritage.

Traduction du modèle en langage SQL

La phase de conception (MLD) a dû être traduite en scripts SQL stricts. La création de la table parente `media` a nécessité l'utilisation de types de données spécifiques pour optimiser le stockage, notamment l'utilisation du type `ENUM` pour distinguer rapidement les films des séries.

Extrait du script de création de la table parente :

SQL

```
CREATE TABLE media (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  type ENUM('film', 'serie') NOT NULL,  
  title VARCHAR(255) NOT NULL,  
  slug VARCHAR(100) NOT NULL UNIQUE,  
  description TEXT,  
  release_date DATE,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

L'ajout de la colonne `created_at` avec la contrainte `DEFAULT CURRENT_TIMESTAMP` permet d'automatiser l'horodatage lors de l'insertion d'un nouveau média. Cette fonctionnalité côté serveur soulage le code PHP, qui n'a plus besoin de générer et d'insérer la date manuellement, garantissant ainsi l'intégrité temporelle des données pour l'affichage des "Nouveautés" sur la page d'accueil.

2. Gestion des enfants et des contraintes

La création des tables enfants (**films** et **series**) a nécessité la mise en place de clés étrangères strictes pointant vers la table parente.

SQL

```
CREATE TABLE films (  
    media_id INT PRIMARY KEY,  
    video_filename VARCHAR(255) NOT NULL,  
    duration INT,  
    FOREIGN KEY (media_id) REFERENCES media(id) ON DELETE CASCADE  
);
```

Dans ce script, la déclaration **ON DELETE CASCADE** est cruciale. Elle assure l'intégrité référentielle de la base : si un administrateur supprime une série de la table **media**, le moteur MySQL supprimera automatiquement l'entrée correspondante dans la table **series**, évitant ainsi la présence de données orphelines. La clé **media_id** agit simultanément comme identifiant unique (Clé Primaire) et comme lien de parenté (Clé Étrangère).

3. Débogage et résolution d'incidents

Problématique de refactorisation

Durant le développement, l'architecture a dû être itérée et modifiée. Lors de la tentative de nettoyage de l'ancienne structure de base de données pour déployer la nouvelle version avec le double héritage, je me suis heurté à un blocage de sécurité de la part de MySQL.

L'erreur **#1451 - Impossible de supprimer un enregistrement père : une contrainte externe l'empêche** se déclenchait car les anciennes tables de liaison (**media_actors**, **favorites**) pointaient encore vers les tables principales. Le moteur transactionnel de MySQL interdisait la destruction des tables pour protéger les données.

Solution apportée

Pour contourner cette sécurité proprement le temps de la mise à jour structurelle, il a fallu élaborer un script de nettoyage forcé. La solution a consisté à désactiver temporairement la vérification des clés étrangères.

SQL

```
-- 1. Désactivation temporaire de la sécurité des clés étrangères
SET FOREIGN_KEY_CHECKS = 0;

-- 2. Suppression en cascade des tables de liaison PUIS des
tables principales
DROP TABLE IF EXISTS favorites, votes, progress;
DROP TABLE IF EXISTS media_actors, media_categories,
media_directors, episodes, films, series, media;
DROP TABLE IF EXISTS users, actors, directors, categories;

-- 3. Réactivation immédiate de la sécurité après le nettoyage
SET FOREIGN_KEY_CHECKS = 1;
```

Cette intervention technique a permis de réinitialiser l'environnement de développement sans avoir à supprimer la base de données entière, démontrant l'importance de maîtriser les variables d'environnement SQL (`FOREIGN_KEY_CHECKS`).

4. Insertion des jeux d'essais et requêtes complexes

Une fois l'architecture déployée, il était nécessaire de la tester avec un jeu de données réel pour s'assurer du bon fonctionnement des liaisons. J'ai donc rédigé les requêtes d'insertion (INSERT INTO) pour plusieurs œuvres complexes, notamment le film "Star Wars: Episode III" avec ses acteurs et réalisateurs associés.

Pour récupérer ces données en PHP et les afficher sur l'interface, l'utilisation de jointures complexes (**JOIN**) était inévitable. Voici par exemple la requête développée pour afficher la fiche détaillée d'un film :

```
SQL
SELECT m.title, m.description, m.release_date, f.video_filename,
f.duration
FROM media m
JOIN films f ON m.id = f.media_id
WHERE m.slug = "star_wars_3";
```

Cette méthode de sélection ciblée permet de ne récupérer que les informations strictement nécessaires pour la page demandée, optimisant ainsi le temps de chargement et l'utilisation de la mémoire du serveur, des critères essentiels pour une plateforme de streaming.

VII. Développement Frontend et Optimisation

1. L'enjeu de l'interface et du référencement naturel (SEO)

Concevoir pour l'utilisateur et pour les moteurs de recherche

Une fois l'architecture de la base de données finalisée et la logique métier opérationnelle en PHP, la dernière grande étape du projet consistait à développer l'interface utilisateur. Cependant, le développement frontend d'une plateforme de streaming moderne ne se limite pas à produire un design attrayant. Il faut répondre à une double exigence : offrir une navigation fluide à l'utilisateur humain, tout en facilitant la lecture du code par les robots d'indexation (comme Googlebot) pour optimiser le référencement naturel (SEO).

Le rôle clé du "Slug" dans la stratégie SEO (URL Rewriting)

C'est ici que l'anticipation lors de la modélisation de la base de données a porté ses fruits. Historiquement, de nombreux sites utilisaient des URLs dynamiques brutes, complexes et illisibles pour les moteurs de recherche (par exemple :

`formatinfo-stream.fr/media.php?type=film&id=42`).

Grâce à la colonne `slug` intégrée dans ma table `media`, j'ai pu mettre en place une stratégie d'URL Rewriting (Réécriture d'URL). Le système PHP génère ainsi des URLs propres, sémantiques et riches en mots-clés :

- `formatinfo-stream.fr/film/star-wars-episode-3`

Ces "Friendly URLs" sont un critère fondamental pour les algorithmes de référencement, car elles indiquent immédiatement le sujet de la page avant même d'en analyser le contenu.

2. L'intégration technique du SEO via PHP

Pour que chaque fiche de film ou de série soit correctement indexée, il ne suffisait pas de coder une seule page HTML statique. J'ai dû concevoir un template (modèle) PHP dynamique capable de modifier les balises `<head>` du HTML en fonction du média appelé depuis la base de données.

Extrait du code d'en-tête dynamique générant les balises SEO essentielles :

```
HTML
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">

  <title><?= htmlspecialchars($media['title']); ?> en streaming
- FORMATINFO VOD</title>

  <meta name="description" content="Regardez <?=
htmlspecialchars($media['title']); ?>. <?=
substr(htmlspecialchars($media['description']), 0, 150); ?>...">

  <link rel="stylesheet" href="assets/css/style.css">
</head>
```

Ce script illustre l'importance du lien entre le SQL et le Frontend. En injectant directement le titre et le début de la description (`substr` limité à 150 caractères pour respecter les standards de Google) dans les balises `<meta>`, chaque page générée devient unique aux yeux des moteurs de recherche, évitant ainsi les pénalités pour contenu dupliqué (Duplicate Content). De plus, l'utilisation systématique de la fonction `htmlspecialchars()` sécurise l'affichage contre les failles XSS, alliant ainsi SEO et cybersécurité.

3. Conception des pages de façade et hiérarchie sémantique

Structure sémantique du contenu (HTML5)

Outre les métadonnées, le corps des pages a été structuré en respectant la sémantique HTML5. Sur la page de détail d'un film, le titre principal de l'œuvre est systématiquement encapsulé dans une balise `<h1>`, tandis que les sections secondaires (comme "Acteurs", "Épisodes similaires") utilisent des balises `<h2>`. Cette hiérarchisation stricte du contenu guide les robots d'indexation pour comprendre l'importance relative de chaque information.

Le design de l'interface (UI)

L'aspect visuel a été pensé pour reproduire l'ergonomie des leaders du marché. J'ai privilégié un thème sombre (Dark Mode) pour mettre en valeur les jaquettes des films et réduire la fatigue visuelle lors de la navigation nocturne.

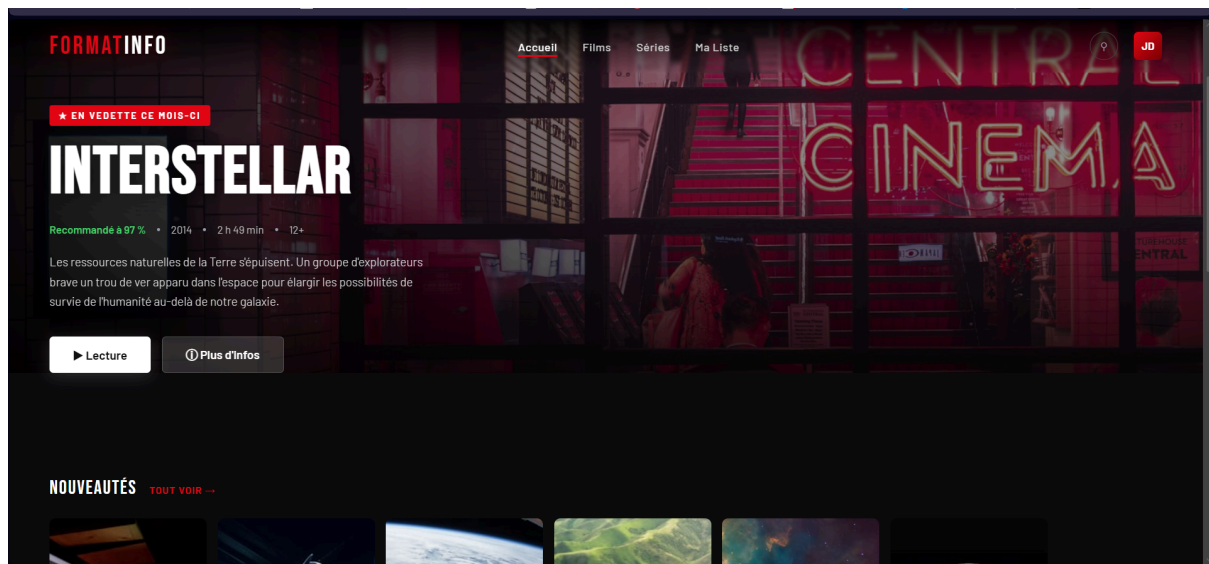


Figure 3 : Page d'accueil dynamique présentant les dernières sorties avec un carrousel horizontal.

4. L'affichage dynamique des médias

La page de lecture vidéo représente l'aboutissement du projet. Elle rassemble toutes les technologies mises en œuvre : la base de données fournit les informations, le PHP route les données, et le HTML/CSS les affiche de manière esthétique.

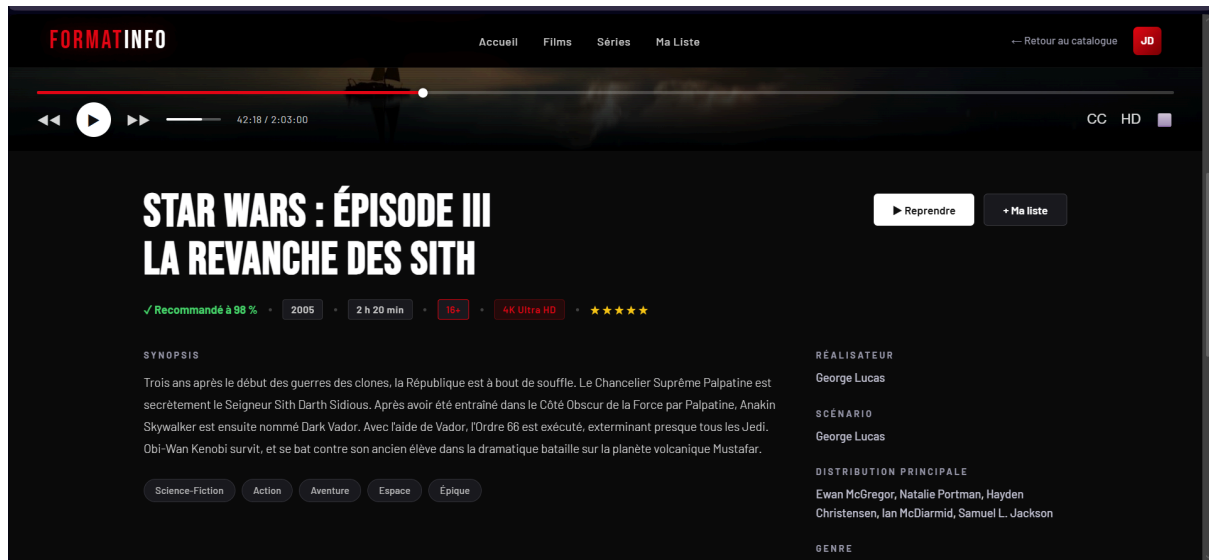


Figure 4 : Interface de lecture et fiche détaillée d'un média, affichant les métadonnées et le lecteur vidéo.

L'interface a été conçue en *Responsive Design* (adaptable sur mobile et tablette) en utilisant des grilles CSS (CSS Grid et Flexbox), garantissant une expérience utilisateur optimale quel que soit l'appareil utilisé.

VIII. Bilan, Compétences et Conclusion

1. Bilan technique et compétences mobilisées

Montée en compétences techniques (Blocs SLAM)

Ce stage de huit semaines chez FORMATINFO a été une véritable immersion dans les exigences du développement professionnel. La conception intégrale de cette plateforme de streaming m'a permis de valider et de consolider de nombreuses compétences du référentiel BTS SIO (parcours SLAM), notamment :

- **Conception et adaptation d'une base de données** : Le passage d'une structure simple à un modèle de "Double Héritage" m'a confronté aux réalités de l'optimisation des données. J'ai appris à anticiper la croissance d'un catalogue et à sécuriser l'intégrité de la base via des clés étrangères strictes.
- **Développement d'une solution logicielle** : La réalisation du backend en PHP natif a renforcé ma maîtrise de la logique algorithmique, des requêtes SQL complexes (**JOIN**, **CASCADE**) et de la sécurité (prévention des failles par échappement des données).
- **Déploiement et intégration** : La mise en place d'une interface frontend responsive et optimisée pour le SEO m'a appris à lier la logique métier à l'expérience utilisateur.

Résolution de problèmes et débogage

L'un des apprentissages majeurs de ce projet a été la gestion des erreurs. Que ce soit pour contourner les contraintes d'intégrité MySQL (Erreur #1451) lors de la refactorisation de la base, ou pour corriger des erreurs de routage dynamique des images, j'ai développé une méthodologie de débogage beaucoup plus rigoureuse et analytique.

2. Bilan professionnel et humain

Autonomie et gestion de projet

Le fait de réaliser ce stage au sein d'une structure individuelle, en lien direct avec le dirigeant fondateur, a constitué un environnement de travail exceptionnel. Contrairement à une hiérarchie classique et parfois lourde, les circuits de validation étaient très courts. Cette configuration m'a imposé de faire preuve d'une grande autonomie : je devais non seulement exécuter les tâches, mais aussi être force de proposition sur l'architecture technique et les outils à déployer.

J'ai appris à structurer mes journées, à prioriser mes développements (commencer par la robustesse de la base de données avant de m'attarder sur le design) et à documenter mes choix techniques pour pouvoir les justifier auprès de mon tuteur.

3. Conclusion et perspectives

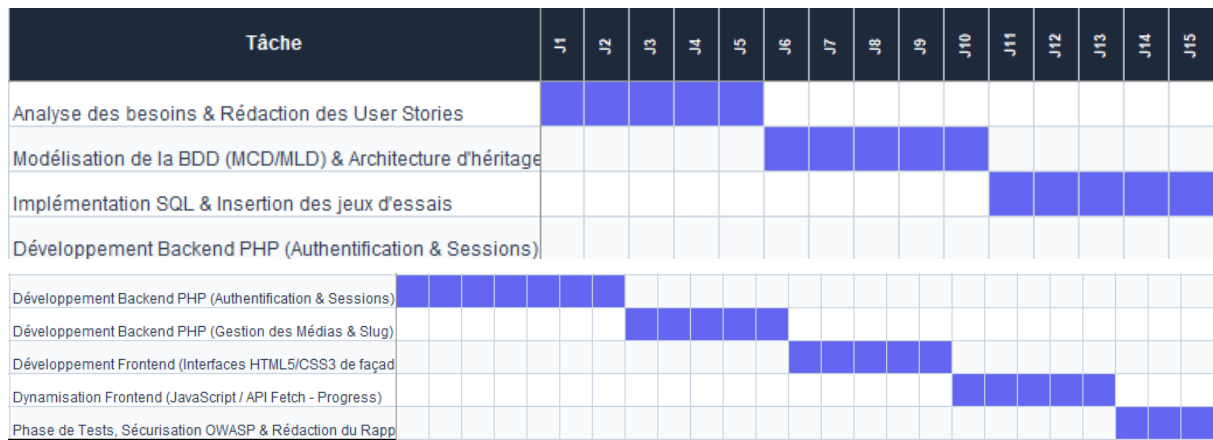
La mission qui m'a été confiée par FORMATINFO était ambitieuse : concevoir l'architecture complète d'une plateforme VOD. Les objectifs ont été atteints, avec la livraison d'un "Proof of Concept" fonctionnel, dont la base de données est prête à absorber un volume de médias important de manière structurée et évolutive.

Au-delà de la réussite technique, ce stage m'a apporté une vision concrète du métier de développeur backend. Il a mis en lumière l'importance cruciale de la phase d'analyse et de modélisation, prouvant qu'un code propre découle avant tout d'une architecture de données bien pensée.

Cette expérience professionnelle confirme ma volonté de poursuivre ma spécialisation dans le développement de solutions web robustes. L'autonomie acquise durant ces huit semaines me donne aujourd'hui la confiance nécessaire pour aborder de nouveaux défis. Je me sens pleinement préparé à intégrer une équipe de développement dans le cadre d'un contrat en alternance dès l'année prochaine, afin de continuer à me confronter à des environnements techniques toujours plus complexes, pourquoi pas en explorant de nouveaux frameworks ou des architectures cloud.

Annexes

Diagramme de Gantt du projet



Dictionnaire de Données (MCD)

Table: media				
Nom du Champ	Type SQL	Taille	Clé	Description / Contraintes
id	INT	11	PK	Identifiant unique du média (Auto-incrément)
type	ENUM	('film', 'serie')		Discriminant d'héritage
title	VARCHAR	255		Titre complet de l'œuvre
slug	VARCHAR	100	UNIQUE	Identifiant URL pour le SEO et le routage des images
description	TEXT	-		Synopsis du média
release_date	DATE	-		Date de sortie officielle
created_at	TIMESTAMP	-		Date d'ajout au catalogue (Générée automatiquement)

Table: films				
Nom du Champ	Type SQL	Taille	Clé	Description / Contraintes

media_id	INT	11	PK, FK	Identifiant hérité de la table media (ON DELETE CASCADE)
video_filename	VARCHAR	255		Nom du fichier vidéo source (ex: star_wars_3.mp4)
duration	INT	11		Durée totale du film en minutes

Table: series				
Nom du Champ	Type SQL	Taille	Clé	Description / Contraintes
media_id	INT	11	PK, FK	Identifiant hérité de la table media (ON DELETE CASCADE)
total_seasons	INT	11		Nombre total de saisons disponibles

Table: episodes				
Nom du Champ	Type SQL	Taille	Clé	Description / Contraintes
id	INT	11	PK	Identifiant unique de l'épisode
serie_id	INT	11	FK	Référence à la série parente (ON DELETE CASCADE)
season_number	INT	11		Numéro de la saison
episode_number	INT	11		Numéro de l'épisode dans la saison
title	VARCHAR	255		Titre spécifique de l'épisode
video_filename	VARCHAR	255		Nom du fichier vidéo de l'épisode
duration	INT	11		Durée de l'épisode en minutes

Table: users				
--------------	--	--	--	--

Nom du Champ	Type SQL	Taille	Clé	Description / Contraintes
id	INT	11	PK	Identifiant unique de l'utilisateur
email	VARCHAR	255	UNIQUE	Adresse email de connexion
password	VARCHAR	255		Mot de passe haché (bcrypt)
created_at	TIMESTAMP	-		Date d'inscription

Table: progress (Reprise de lecture)

Nom du Champ	Type SQL	Taille	Clé	Description / Contraintes
user_id	INT	11	PK, FK	Identifiant de l'utilisateur
media_id	INT	11	PK, FK	Identifiant du média visionné
episode_id	INT	11	FK	Identifiant de l'épisode (NULL si c'est un film)
timestamp	INT	11		Temps de lecture sauvegardé en secondes
updated_at	TIMESTAMP	-		Dernière mise à jour de la progression

Script Backend (API de reprise de lecture)

Fichier : `api/update_progress.php`

Ce script PHP agit comme un point de terminaison (API) invisible pour l'utilisateur. Il est appelé toutes les 10 secondes par le JavaScript du lecteur vidéo en arrière-plan. Il sécurise la donnée, vérifie l'identité de l'utilisateur, et met à jour son temps de visionnage dans la base de données pour lui permettre de reprendre son film plus tard.

```
PHP
<?php
// 1. Initialisation de la session et sécurisation
session_start();

// Définition de l'en-tête pour renvoyer du JSON au JavaScript
header('Content-Type: application/json');

// Vérification : L'utilisateur doit être connecté pour
sauvegarder sa progression
if (!isset($_SESSION['user_id'])) {
    http_response_code(401); // Erreur : Non autorisé
    echo json_encode(["status" => "error", "message" =>
"Utilisateur non authentifié."]);
    exit();
}

// 2. Récupération des données asynchrones (Fetch API)
// Le JavaScript envoie les données au format JSON dans le corps
de la requête
$json_data = file_get_contents("php://input");
$request = json_decode($json_data, true);

// 3. Nettoyage et validation des variables
$user_id    = $_SESSION['user_id'];
$media_id   = filter_var($request['media_id'],
FILTER_VALIDATE_INT);
$timestamp  = filter_var($request['timestamp'],
FILTER_VALIDATE_INT);

// Gestion de l'héritage : l'episode_id peut être NULL (si c'est
un film)
```

```

$episode_id = isset($request['episode_id']) ?
filter_var($request['episode_id'], FILTER_VALIDATE_INT) : null;

// Si les identifiants ou le temps sont invalides, on stoppe
l'exécution
if (!$media_id || $timestamp === false) {
    http_response_code(400); // Erreur : Mauvaise requête
    echo json_encode(["status" => "error", "message" => "Données
invalides."]);
    exit();
}

try {
    // 4. Connexion sécurisée à la base de données (PDO)
    $pdo = new
PDO('mysql:host=localhost;dbname=formatinfo_vod;charset=utf8mb4',
'root', '');
    $pdo->setAttribute(PDO::ATTR_ERRMODE,
PDO::ERRMODE_EXCEPTION);

    // 5. Requête SQL complexe : "Insérer ou Mettre à jour"
    // Si l'utilisateur regarde ce média pour la première fois,
on crée la ligne.
    // S'il l'avait déjà commencé (la clé primaire existe), on
met juste à jour le temps.
    $sql = "INSERT INTO progress (user_id, media_id, episode_id,
timestamp)
VALUES (:user_id, :media_id, :episode_id, :timestamp)
ON DUPLICATE KEY UPDATE
timestamp = :timestamp,
updated_at = CURRENT_TIMESTAMP";

    $stmt = $pdo->prepare($sql);

    // 6. Exécution de la requête préparée (Protection contre les
Injections SQL)
    $stmt->execute([
        ':user_id' => $user_id,
        ':media_id' => $media_id,

```

```

        ':episode_id' => $episode_id,
        ':timestamp'   => $timestamp
    ]);

    // 7. Réponse de succès renvoyée au frontend
    echo json_encode(["status" => "success", "message" =>
"Progression sauvegardée avec succès à {$timestamp}s."]);

} catch (PDOException $e) {
    // Gestion des erreurs liées à la base de données
    http_response_code(500); // Erreur : Problème serveur
    echo json_encode(["status" => "error", "message" => "Erreur
lors de la sauvegarde."]);
}
?>

```